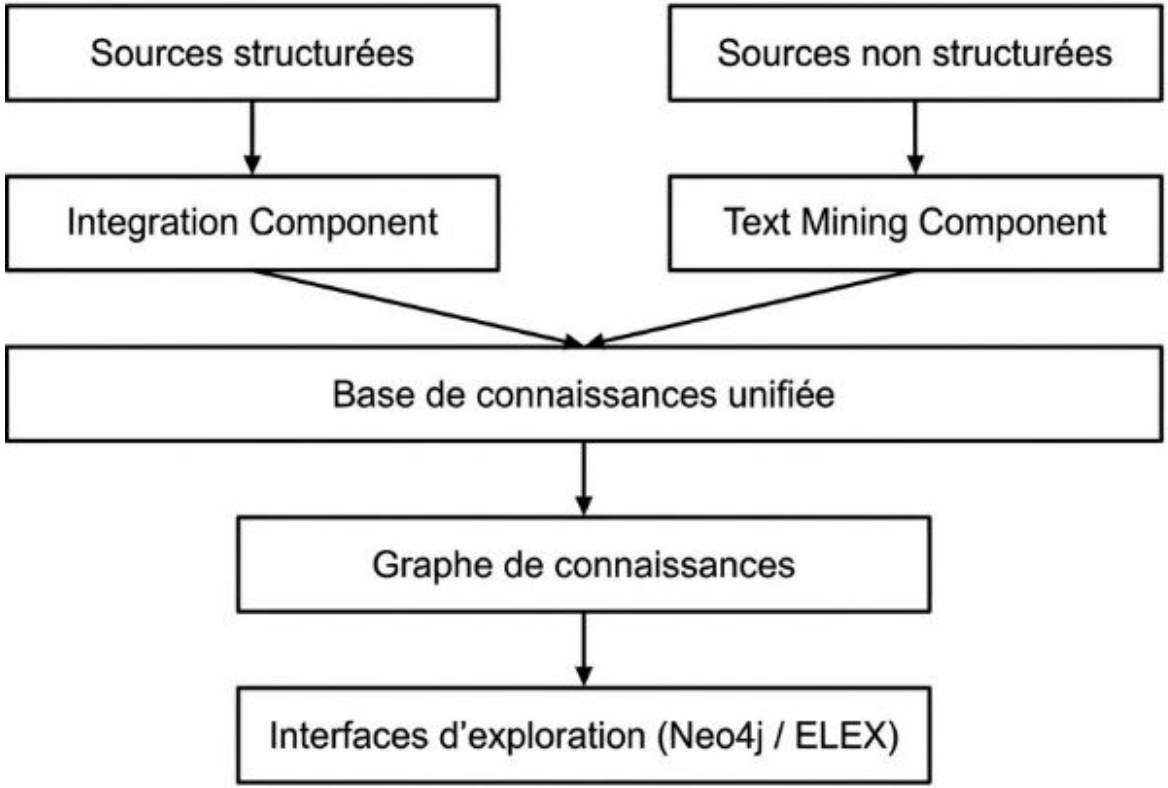
Construction/Peuplement de graphes de connaissances

Nour CHERIF
Lina CHENNI
Jennifer SAID

Plan:

1. Etude des articles
2. Présentation de notre solution
3. Démo

Article 1 — CurEx (Loster & Naumann, 2018)

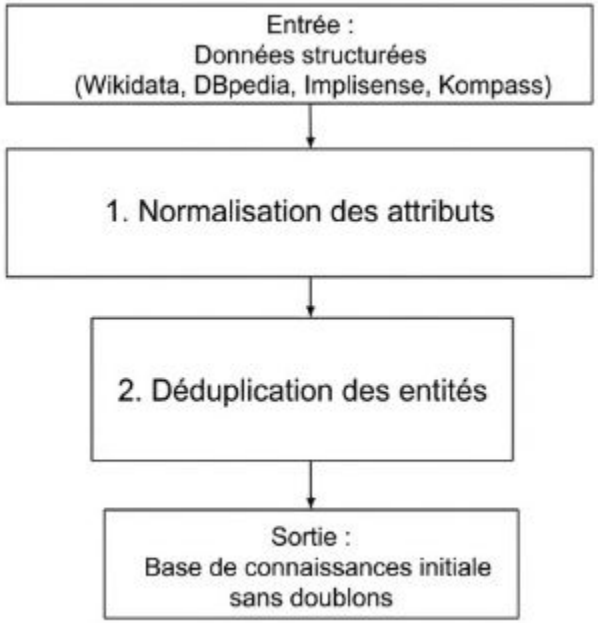


CurEx propose un système permettant d'automatiser une grande partie de ce processus.

Il repose sur une architecture modulaire et scalable basée sur Apache Spark et Cassandra.

Branche 1 : Intégration des données structurées

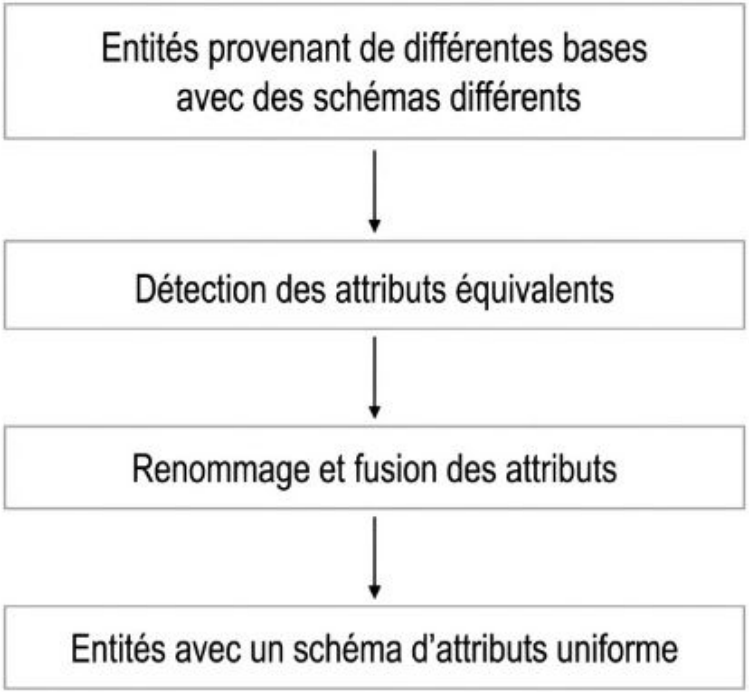
Ce pipeline vise à créer une base de connaissances initiale.



Étape 1 : Normalisation

Objectif

Uniformiser les attributs sémantiquement identiques provenant de différentes sources et les fusionner si nécessaire.



Étape 2 : Déduplication

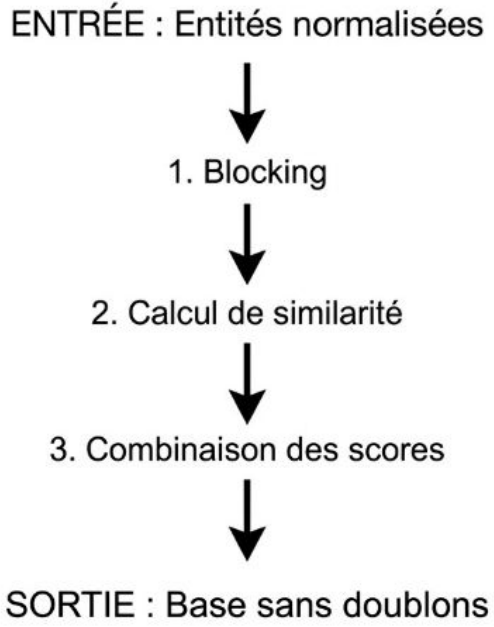
Objectif: Fusionner les entités représentant le même objet réel.

Blocking (pour min les comparaisons): Regrouper les entités en blocs plus petits selon une clé commune (code postal par ex). Comparaisons dans chaque bloc.

Calcul de similarité: Pour chaque paire d'entités, on compare un sous-ensemble d'attributs (noms, id, …) avec des mesures de similarité (non précisées).

Combinaison des résultats linéairement en un score normalisé entre 0 et 1.
Si: score > seuil → doublon détecté, fusion des entités.

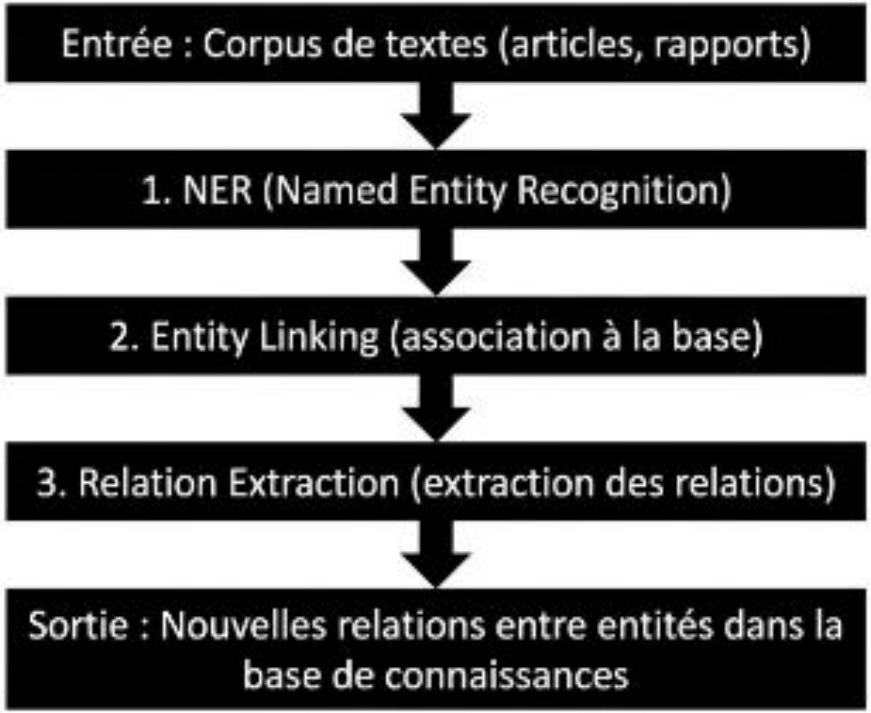
Résultats après déduplication: 2,1 millions d'entités finales, 29 000 entités présentes dans exactement 2 sources, 3 700 présentes dans 3 sources.



Branche 2 : Text Mining

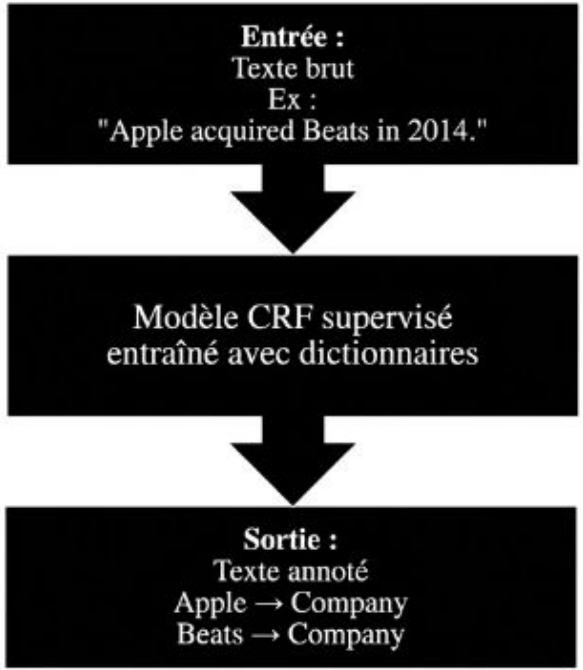
Objectif : Extraire de nouvelles informations depuis des **textes non structurés** pour enrichir la base.

(Le corpus de texte en entrée a subi une phase de pré-traitement non expliquée dans l'article)



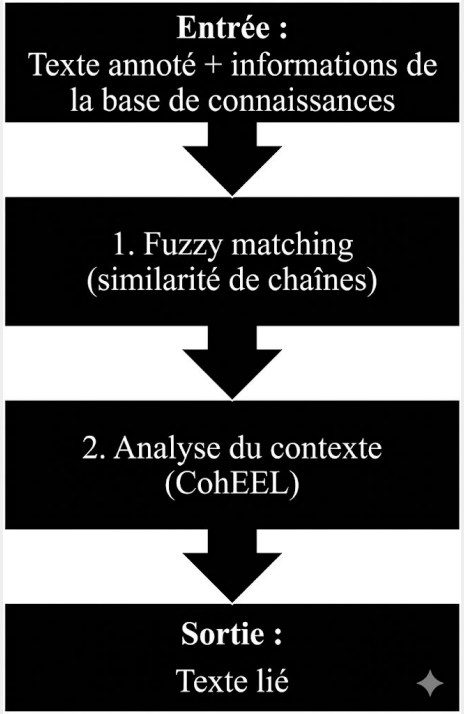
Étape 1 : Named Entity Recognition (NER)

Objectif: identifier les entités dans le texte (entreprises, organisations, personnes, …)



Étape 2 : Entity Linking (EL)

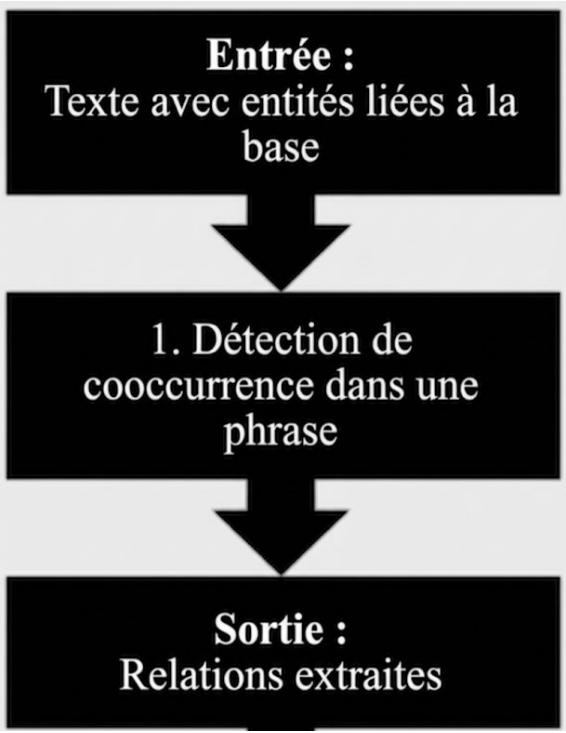
Problème: une entité extraite par NER peut faire référence à plusieurs entrées possibles dans la base de connaissances.
Exemple : Apple peut être: Apple Inc, Apple Records



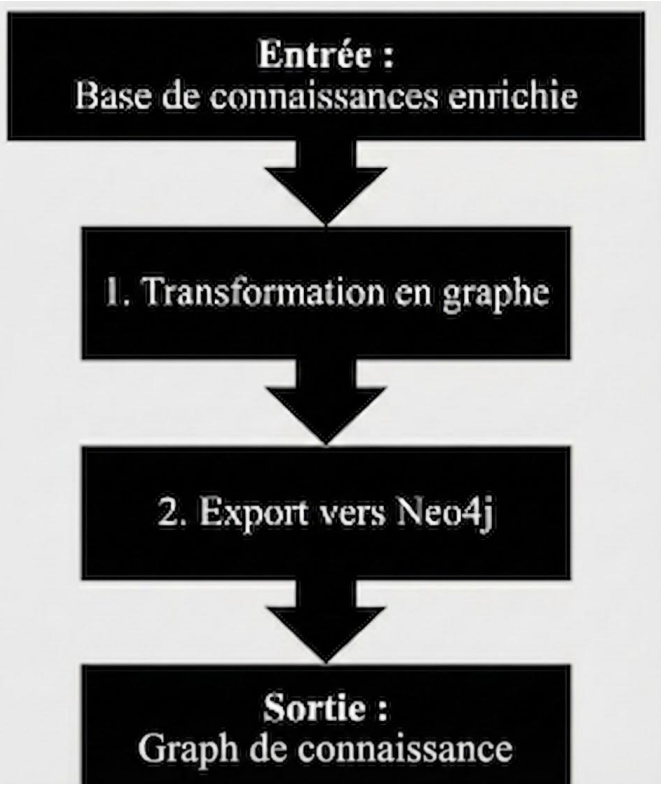
Étape 3 : Relation Extraction (RELEX)

Objectif : détecter les relations entre les entités précédemment découvertes.

-> ce composant peut être remplacé par des techniques plus avancées (réseaux de neurones) pour détecter des relations directionnelles



Construction du graphe



Interfaces

Interface 1 : Curation Interface

Pour les **data engineers**

- contrôler le pipeline
- inspecter les doublons
- modifier directement les entités
- évaluer les performances (précision, rappel, score F1 de la mesure de similarité utilisée)
- gérer les versions

Interface 2 : Entity Landscape Explorer (ELEX)

Pour les **utilisateurs finaux**

- explorer le graphe
- rechercher des entités
- filtrer les relations
- visualiser les sous-graphes
- signaler des erreurs

Avantages de l’approche

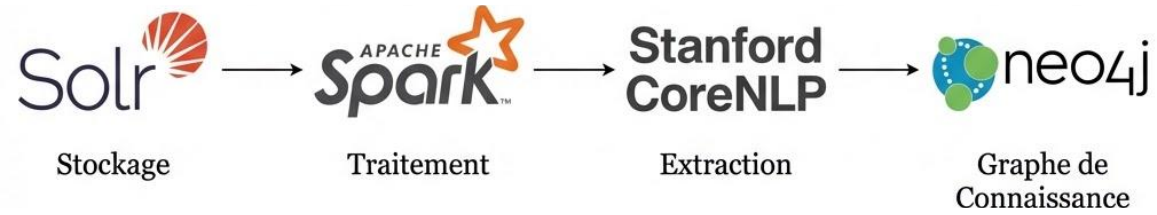
- Intégration de données structurées et non structurées
- Architecture modulaire
- Scalabilité avec Apache Spark
- Automatisation de tâches complexes
- Interfaces adaptées aux utilisateurs

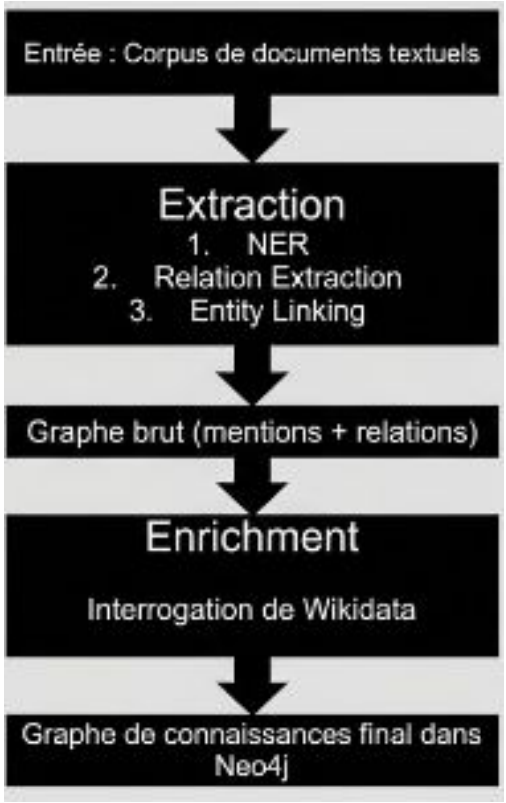
Limites de l’approche

- Relations basées sur cooccurrence peu précises
- Propagation d’erreurs possible :
  NER → EL → RELEX
- Dépendance à la qualité des données
- Besoin de supervision humaine

Article 2: Knowledge Graph Construction from Unstructured Text with Applications to Fact Verification and Beyond

Objectif: dstlr est une plateforme open-source, end-to-end capable de transformer un corpus de textes en un graphe de connaissances puis d'enrichir ce graphe à l'aide d'un graphe de connaissances externe (Wikidata).





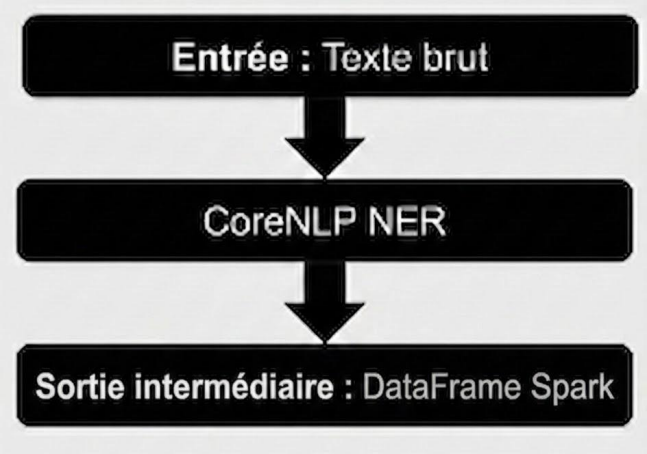
Extraction - Étape 1 : Named Entity Recognition (NER)

Objectif : Identifier les entités dans les documents.

L'outil: NERClassifierCombiner de Stanford CoreNLP qui détecte 20 types d'entités: personnes, organisations, lieux, …

Pour chaque mention (segment de texte) détectée, il enregistre l'id du doc, l'id de la mention, le type d'entité, le texte exact et ses décalages de caractères.

Fusion des mentions identiques dans un même document

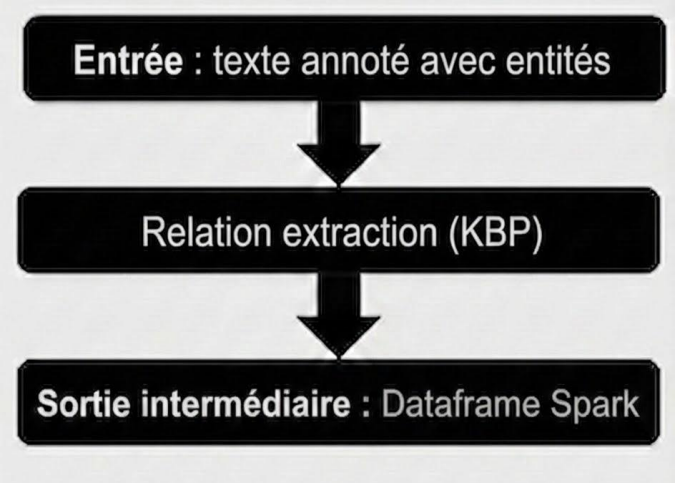


Extraction - Étape 2 : Extraction de relations

Objectif : Identifier les relations entre entités. CoreNLP propose 2 annotateurs: OpenIE et KBP.

Les auteurs utilisent **KBP**. L'outil analyse les phrases pour trouver des liens entre deux mentions, en se limitant strictement à un catalogue de 45 types de relations fixes. Cela facilite l'alignement avec Wikidata.

-> *L'article ne détaille pas les règles linguistiques ou statistiques qui permettent à l'outil de valider une relation*



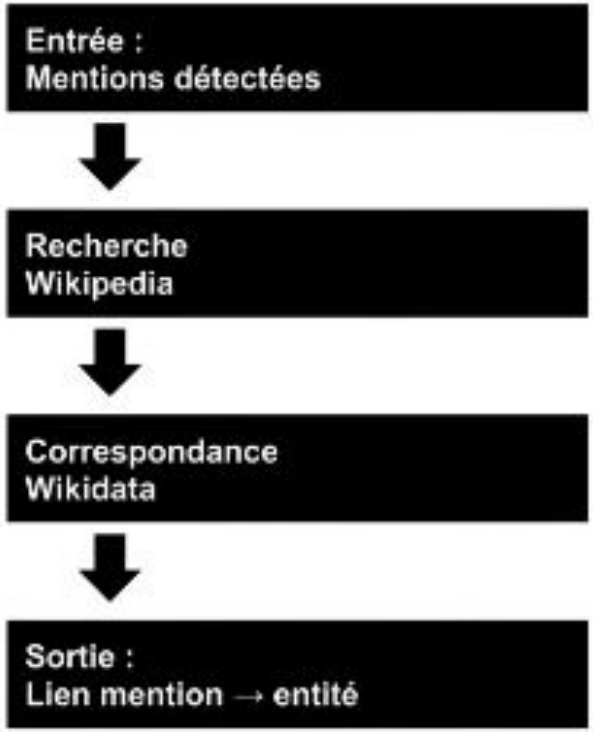
Extraction - Étape 3 : Entity Linking

Objectif : Relier les mentions extraites à leur entité Wikipédia correspondante pour ensuite les associer à des entités Wikidata.

Le système utilise WikidictAnnotator de CoreNLP. Même si aucun lien n’est trouvé la mention est conservée.

Cette étape est essentielle car elle relie le texte au référentiel externe et rend possible l’enrichissement ultérieur.

L'article n'explique pas comment l'outil gère la désambiguïsation (par ex: comment il sait si "Apple" désigne le fruit ou l'entreprise).



Synthèse de l'Extraction: exemple

- **Le texte source** (Doc.*b2c9a*) : “Facebook is an American social media and social networking company based in Menlo Park”

- **Résultats des 3 pipelines d'extraction :**
  - **Mentions (NER) :** « Facebook » et « Menlo Park ».
  - **Relation (KBP) :** has_hq (possède un siège social).
  - **Lien Wikidata (Entity Linking) :** « Facebook » -> Entité Q355.

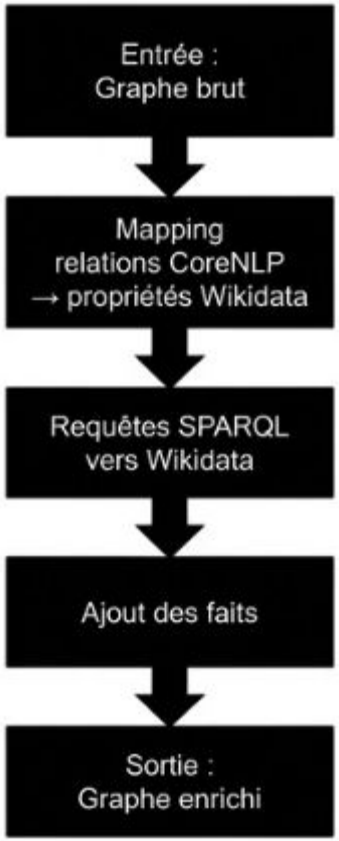
- **Le résultat final** (graph sous forme de triplets) :
  - (*b2c9a*, mentions, « Facebook »)
  - (*b2c9a*, mentions, « Menlo Park »)
  - (« Facebook », sujet_de, *has_hq*)
  - (*has_hq*, objet_de, « Menlo Park »)
  - (« Facebook », liens_vers, *Q355*)

Phase 2 : Enrichissement

Objectif : Compléter le graphe brut avec des informations provenant de **Wikidata**.

1) définir manuellement une correspondance entre les relations CoreNLP et les propriétés Wikidata (ex: la relation « siège social » de CoreNLP correspond à P159 dans Wikidata.)

2) Pour chaque entité distincte détectée dans le corpus, le système interroge Wikidata via SPARQL. Les faits (triplets) correspondants sont extraits et insérés dans Neo4j. Seules les entités réellement présentes dans le corpus sont enrichies, ce qui optimise stockage et performances.



Applications : L'alignement de sous-graphes

Objectif : Comparer le graphe extrait du texte avec le graphe externe (Wikidata) via des requêtes Cypher.

Le système prend un fait extrait du texte et regarder s'il trouve son équivalent exact dans les données de Wikidata. 3 cas de figure possibles:

1) Relation extraite = Wikidata -> fait confirmé
2) Relation extraite ≠ Wikidata -> incohérence détectée
3) Relation extraite sans fait Wikidata -> nouveau fait potentiel

Avantages de l’approche

- Pipeline complet et intégré de bout en bout
- Scalabilité
- Conservation de la provenance des informations
- Intégration directe avec un graphe externe
- Plateforme open-source

Limites de l’approche

- Dépendance au module KBP de CoreNLP (ensemble limité de 45 relations)
- Mapping manuel entre relations et propriétés Wikidata
- Pas d’utilisation de modèles neuronaux récents pour l’extraction
- Dépendance à la qualité de Wikidata
- Les erreurs d’extraction peuvent se propager dans le graphe

Article 3: Automatic Ontology Population from Textual Advertisements: LLM vs. Semantic Approach

Context

Internet contient énormément de textes; le problème est que ces textes sont souvent :

- Courts
- Mal structurés
- Incomplets
- Ecrits pour attirer l'attention plutôt que pour expliquer clairement.

Exemple d'annonce :

"Maison 120m², 5 chambres, jardin, proche Caen".

Pour un humain c'est clair, mais pour un ordinateur c'est difficile à comprendre.

L'objectif de l'article est donc :

Transformer automatiquement ces textes en informations structurées que l'ordinateur peut comprendre.

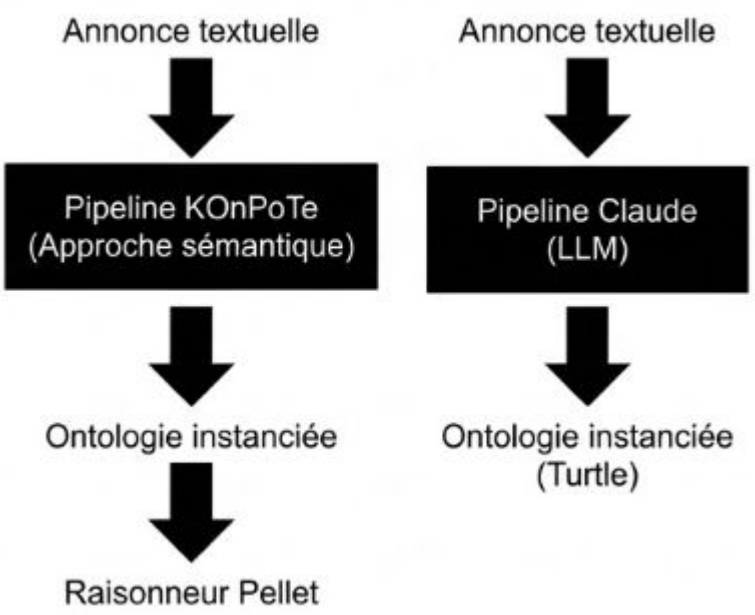
Pipeline global de l’étude

Méthode 1 : Approche sémantique (KOnPoTe)

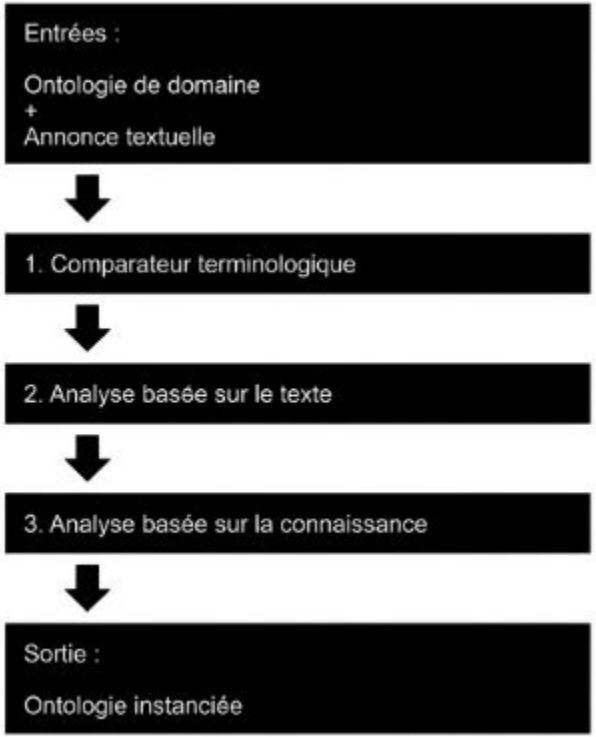
1. Comparer texte / ontologie
2. Analyser le texte
3. Utiliser les règles de l’ontologie pour compléter.

Méthode 2 : Approche avec IA (LLM)

1. Envoyer prompt + ontologie + texte
2. Le LLM comprend le texte
3. Il génère directement les triples.

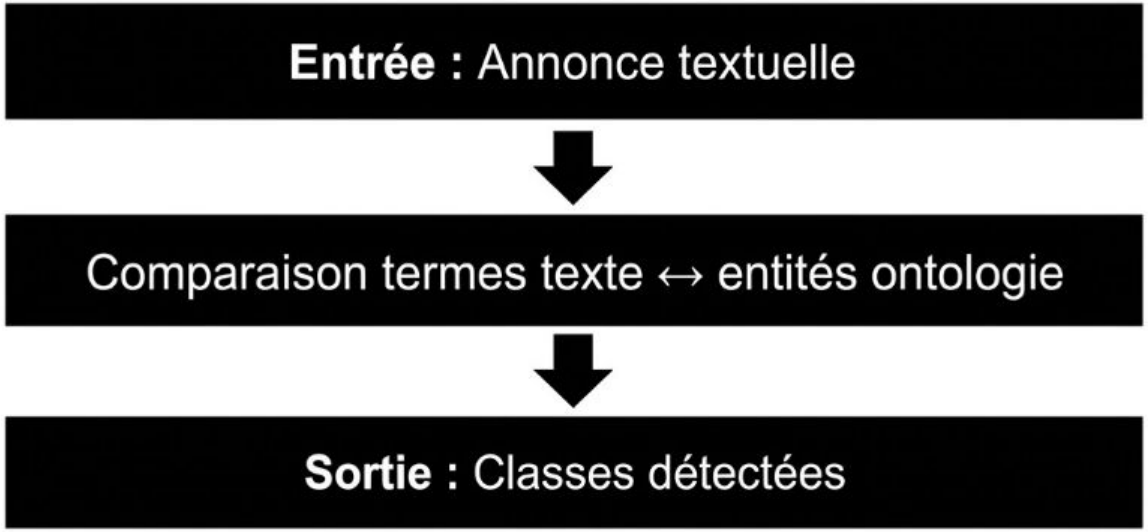


Pipeline KOnPoTe



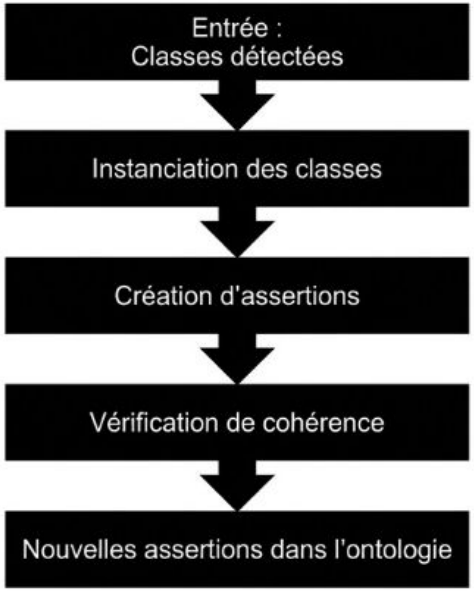
Étape 1 : Comparateur terminologique

Objectif :
Identifier les **termes du texte correspondant aux entités de l'ontologie**.



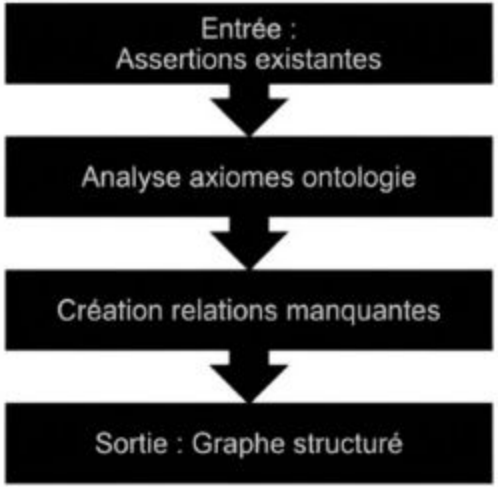
Étape 2 : Analyse basée sur le texte

Objectif :
Créer des instances et relations à partir du texte.



Étape 3 : Analyse basée sur la connaissance

Objectif :
Exploiter les axiomes ontologiques



Avantages KOnPoTe

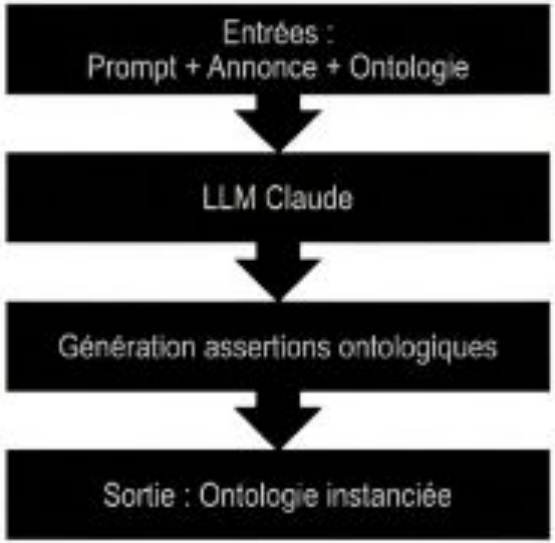
- Cohérence logique
- Respect des axiomes
- Gestion des relations n-aires

Limites

- Individus redondants
- Difficultés avec certains textes
- Dépendance aux termes présents dans l’ontologie

Pipeline approche LLM : Claude

Objectif :
Utiliser un LLM pour générer directement une ontologie instanciée.



Prompt Engineering

Les auteurs testent **4 versions**.

Prompt 1
Prompt simple
Résultat :
format non exploitable

Prompt 2
Ajout format Turtle
Résultat :
axiomes non respectés

Prompt 3
Ajout contraintes logiques
Résultat :
hallucinations

Prompt 4
Ajout mot : Exclusively
Résultat :
moins d'hallucinations
Prompt retenu pour l'évaluation.

Avantages :

- Flexibilité linguistique
- Détection d’abréviations
- Peu de redondance
- Pipeline simple

Limites :

- Hallucinations
- Incohérences logiques
- Difficultés avec relations n-aires
- Dépendance au prompt engineering.

Conclusion

Résultat global : La méthode sémantique (KOnPoTe) est meilleure que le LLM.

Méthode	Precision	Recall	F1
KOnPoTe	0.94	0.90	0.92
Claude	0.77	0.59	0.66

Cela signifie que :

- Le LLM fait plus d'erreurs
- Oublie plus d'informations.

➢ L'approche sémantique est plus fiable

➢ mais les LLM sont plus faciles à utiliser.

La meilleure solution serait une approche hybride qui combine :

- LLM pour extraire l'information
- raisonnement ontologique pour corriger les erreurs

Article 4: A Short Review for Ontology Learning: Stride to Large Language Models Trend

Objectif de l’article

- Organiser les connaissances pour que les ordinateurs puissent **comprendre les informations et les relations entre les concepts**.
- Très important dans le domaine comme IA , web sémantique.

Qu'est-ce qu'une ontologie ?

Une **ontologie** est une représentation structurée des connaissances dans un domaine.

Elle contient : concepts, relations et règles.
Exemple d'ontologie

Concepts : médecin, patient, maladie
Relation : médecin soigne patient
Hiérarchie : médecin est une personne

Problème

Création manuelle d’une ontologie

- Analyse de nombreux documents
- Identification des concepts
- Définition des relations entre concepts

Problèmes :

- Processus long
- Nécessite des experts

➡ **Solution : Ontology Learning**
Création automatique d’ontologies à partir de textes

Pipeline global de l’apprentissage d’ontologie

Qu'est-ce que l'Ontology Learning ?

C'est le processus qui permet de créer automatiquement une ontologie en analysant des textes.
Les 6 étapes principales

- Extraction des termes(mots important)
- Trouver les synonymes
- Création des concepts
- Relations hiérarchiques
- Autres relations
- Règles logiques

Les méthodes utilisées:

Manuel : experts créent l'ontologie
Semi-automatique : humain + machine
Automatique : seulement la machine

Pipeline des approches traditionnelles (Shallow Learning)

Les premières méthodes reposent sur :

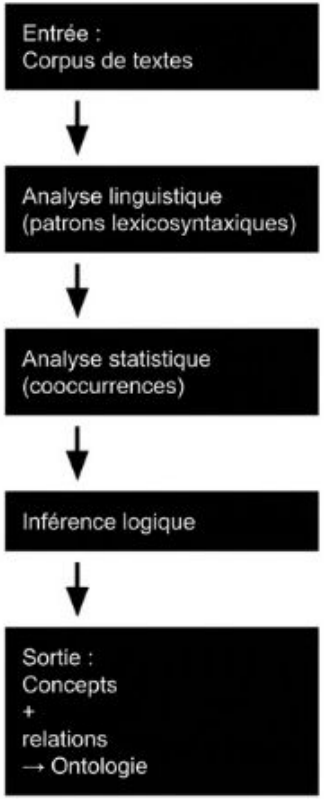
- Linguistique
- Statistiques
- Logique formelle.

Avantages:

- Simplicité
- Rapidité
- Faible besoin en données.

Limites:

- Faible capacité d'apprentissage automatique
- Dépendance à l'intervention humaine
- Difficulté avec les relations complexes
- Mauvaise gestion des grands corpus.



Pipeline Deep Learning

Avec l'essor des réseaux neuronaux, l'extraction devient plus précise.
Modèles utilisés :

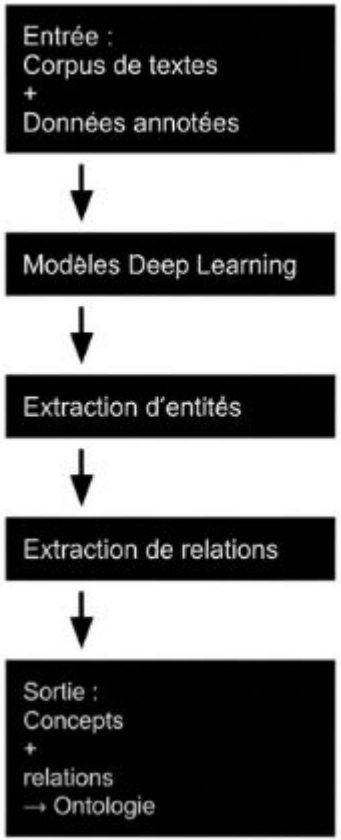
- CNN
- BiLSTM
- BERT

Avantages:

- Meilleur analyse du texte /relation
- Détection de relations plus complexes
- Capacité à traiter de grands corpus.

Limites:

- Besoin de données annotées
- Coût computationnel élevé
- Difficulté d'adaptation à un nouveau domaine
- Ne garantit pas la cohérence logique de l'ontologie.



Pipeline basé sur les LLM

Les LLM peuvent :

- Concept extraction
  - identifier les concepts d'un domaine.
- Extraction hiérarchique
  - identifier relations :
  - est un type de
- Relations non taxonomiques

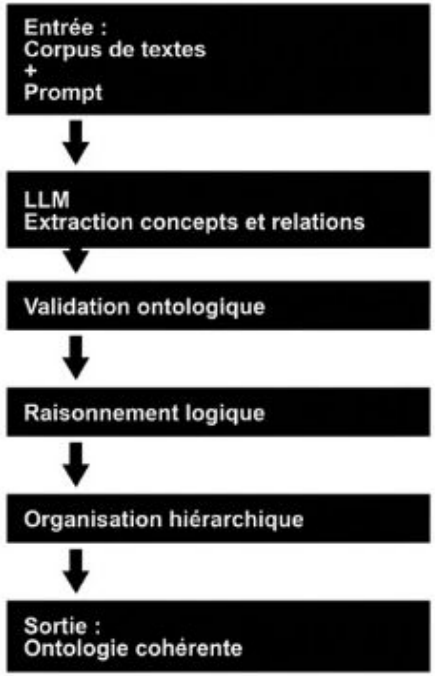
Cependant ces relations restent plus difficiles à extraire.

Avantages:

- Extraction sans données annotées (zero-shot)
- Grande flexibilité linguistique
- Génération automatique d'ontologies
- Capacité à exploiter les connaissances du pré-entraînement.

Limites:

- Résultats parfois instables
- Hallucinations possibles
- Difficulté avec les relations complexes
- Absence de garantie de cohérence logique.



Perspectives pour le futur

- améliorer l’extraction des relations complexes
- collaborer les LLMs avec des experts du domaine
- utiliser les LLMs pour mettre à jour les ontologies
- développer de meilleurs benchmarks

Notre solution

- Les textes contiennent beaucoup d’informations difficiles à exploiter automatiquement.
- Les graphes de connaissances permettent de structurer ces informations sous forme d’entités et de relations.
- L’objectif de ce projet est de transformer automatiquement un corpus textuel en graphe de connaissances.
- Pour cela, nous avons mis en place un pipeline de traitement en plusieurs modules permettant d’extraire, valider et structurer les connaissances.

Pipelines

Corpus texte
↓
Prétraitement
↓
Extraction des phrases
↓
NER (entités)
↓
Entity Linking
↓
Extraction des relations
↓
Normalisation
↓
Validation
↓
Construction du graphe
↓
Graph.json

Module de prétraitement du corpus (preprocessing)

Ce module sert à préparer le texte.

Au départ, le fichier d'entrée contient du texte brut.

Ce texte peut avoir :

- des retours à la ligne,
- des espaces en trop,
- plusieurs phrases mélangées.

Le rôle de ce module est donc de :

- lire le fichier texte,
- nettoyer le texte,
- découper le texte en phrases.

À la fin, il produit une liste de phrases propres.

Chaque phrase a :

- un identifiant de document,
- un identifiant de phrase,
- le texte de la phrase.

Donc, ce module transforme :
texte brut → phrases structurées

Module d’extraction des entités nommées (NER)

Ce module sert à repérer les entités importantes dans les phrases.

Une entité, c’est par exemple :

- une personne,
- une ville,
- un pays,
- une organisation,
- une date.

Le module prend en entrée les phrases produites par preprocessing.py.
Ensuite, il analyse chaque phrase avec spaCy pour détecter les entités.
Pour chaque entité trouvée, il garde :

- la phrase d’origine,
- le mot ou groupe de mots détecté,
- son type,
- sa position dans la phrase.

Donc, ce module transforme :
phrases structurées → liste d’entités détectées

Module d’entity linking

Par exemple, si le texte contient Albert Einstein, ce module essaie de trouver la vraie entité correspondante dans Wikidata.
Il prend en entrée la liste des entités détectées par ner.py.
Pour chaque entité, il :

- nettoie la mention,
- adapte éventuellement le type,
- envoie une recherche à Wikidata,
- récupère si possible :
  - un identifiant,
  - un nom,
  - une description,
  - une URL.

Donc, ce module transforme :
entités détectées → entités enrichies avec des infos externes
Son rôle est important parce qu’il rend les entités plus précises et moins ambiguës

Module d’extraction des relations

Ce module sert à chercher les relations dans les phrases.

Une relation, c’est un lien entre deux éléments.

Par exemple :

- une personne est née à un endroit,
- une personne a étudié une discipline,
- une personne a travaillé quelque part,
- une personne est mariée à quelqu’un.

Ce module prend en entrée les phrases du corpus.
Il lit les phrases et cherche des formes bien précises grâce à des règles.

Quand il reconnaît un motif, il extrait :

- le sujet,
- la relation brute,
- l’objet,
- la phrase source.

Exemple :

- sujet : Albert Einstein
- relation brute : est né à
- objet : Ulm

Donc, ce module transforme :
phrases structurées → relations candidates
On dit “candidates” parce qu’à ce stade elles ne sont pas encore validées.

Module de normalisation des relations

Ce module sert à rendre les relations plus propres et plus uniformes.

Pourquoi ?

Parce que dans le texte, la même idée peut être écrite de plusieurs façons.
Par exemple :

- “est né à”
- “est né dans”
- “lieu de naissance”

Tout ça peut vouloir dire la même chose dans le graphe.
Le rôle du module est donc de prendre une relation brute et de la convertir en une relation standardisée.
Par exemple :

- est né le → birthDate
- est né à → bornInCity
- a travaillé à → worksAt

Il prend donc en entrée les relations candidates extraites auparavant.
En sortie, il produit :

- la relation brute d’origine,
- la relation normalisée,
- une justification,
- une preuve textuelle,
- un statut.

Donc, ce module transforme :
relations candidates → relations normalisées

Module de validation des relations

Ce module sert à vérifier si les relations sont correctes.

C'est une étape très importante.
Le système ne doit pas ajouter n'importe quelle relation dans le graphe final.
Le validateur prend en entrée les relations normalisées.
Ensuite, il vérifie plusieurs choses :

- est-ce que la relation fait partie des relations autorisées ?
- est-ce que le sujet a le bon type ?
- est-ce que l'objet a le bon type ?
- est-ce que la preuve textuelle est bien présente dans la phrase ?

Par exemple :

- birthDate doit relier une personne à une date,
- marriedTo doit relier une personne à une autre personne,
- worksAt doit relier une personne à une organisation.

Si tout est correct, la relation est acceptée.
Sinon, elle est rejetée avec une raison.
Donc, ce module transforme :
relations normalisées → relations validées ou rejetées

Module de construction du graphe

Ce module sert à construire le graphe final.

Il prend en entrée les relations validées et garde seulement les relations acceptées.

À partir de là, il crée :

- des nœuds pour les entités,
- des arêtes pour les relations entre les entités.

Le module fait aussi un autre travail important :

- il évite de créer deux fois le même nœud,
- il donne un type à chaque nœud,
- il crée un journal d'audit pour garder la trace de ce qui a été accepté ou rejeté.

En plus, il peut exporter le résultat :

- en JSON,
- en RDF Turtle,
- en script Cypher pour Neo4j.

Donc, ce module transforme :
relations validées → graphe final + audit log

Module de visualisation du graphe

Ce module sert à **afficher le graphe sous forme visuelle**.

Il prend en entrée le fichier graph.json créé par graph_builder.py.
Ensuite, il utilise Graphviz pour :

- dessiner les nœuds,
- dessiner les relations,
- mettre des couleurs selon le type d'entité.

Par exemple :

- personne en bleu,
- ville en vert,
- organisation en orange.

Le but est simplement de rendre le graphe lisible visuellement.
Donc, ce module transforme :
graphe JSON → image du graphe

Conclusion

Construction automatique d’un graphe de connaissances à partir d’un texte

Pipeline structuré :

- Prétraitement
- Extraction d’entités
- Entity linking
- Extraction des relations
- Normalisation
- Validation
- Construction du graphe

Architecture modulaire et évolutive
Validation + journal d’audit pour garantir la fiabilité et la traçabilité

Base solide pour des applications de web sémantique et analyse de textes

Améliorations possibles :

- relations plus complexes
- intégration avancée d’un LLM